

Envs-FORGE: Frontier-Optimized Reward-Grounded Environment Synthesis for Agent RL

Xiaojun Wu^{*1,2}, Cehao Yang^{*1,2}, Honghao Liu^{*1,2}, Xueyuan Lin^{*2}
 ZhiChao Shi^{1,3}, Hao Zhou^{1,3}, Xuhui Jiang^{1,3}, Chengjin Xu^{1,3}
 Jia Li^{†2}, Jian Guo^{†1}

¹IDEA Research

²The Hong Kong University of Science and Technology (Guangzhou)

³DataArcTech Ltd.

Abstract

Reinforcement learning (RL) for terminal agents needs executable training environments with reliable rewards and useful difficulty. Fixed recipes such as few-shot, Self-Instruct, and Evol-Instruct apply the same prompting policy to every seed, even when the current policy would benefit from a harder, easier, or simply different task. We present Envs-FORGE, a prompting policy that converts verifier rewards into per-seed environment-synthesis actions. Envs-FORGE estimates seed pass rates, scores six projection-direction actions around a target learning frontier, and solves a per-seed mixed-integer linear program (MILP) to choose the action that conditions generation. The selected action drives synchronized rewriting of the instruction, fixtures, oracle solution, tests, and Docker environment; only gold-verified bundles enter RL training. The indexed MILP form also supports optional soft skill coverage for portfolio planning. On Qwen 3.5 35B, Envs-FORGE improves Pass@1 over Base by 9.2 percentage points on tb-core (40.0% to 49.2%) and 6.4 points on tb-2.0 (23.0% to 29.4%), exceeding the strongest fixed-recipe baseline by 2.4 and 2.1 points. It reaches 77.1% on SWE-bench Verified versus 73.4% for Base, and improves tb-core by 6.8–9.2 points across the evaluated 4B–35B models. All synthesis methods export 100 verified environments and use 2.27M–2.88M synthesis tokens, placing the comparison at the same downstream training-set size and the same operational scale. The source code is available at <https://github.com/DataArcTech/DataArc-SynData-Toolkit/>.

and command-line data work. SWE-agent and OpenHands provide interactive execution loops, and Terminal-Bench and SWE-bench Verified evaluate containerized terminal tasks and repository repair (Yang et al., 2024; Wang et al., 2025; Merrill et al., 2026; OpenAI, 2024). RL training for these agents needs more than prompts or transcripts: it needs executable environments whose tests provide reliable rewards and whose difficulty matches the current policy.

We study the prompting policy for environment synthesis. Few-shot, Self-Instruct, Evol-Instruct, and agentic prompting flows define how an LLM generates or rewrites tasks (Wang et al., 2023; Xu et al., 2025; Mitra et al., 2024). Terminal-synthesis systems add useful auxiliary components, including healthy-runtime inversion, skill-graph sampling, error injection, repository construction, procedural generation, and capability-gap discovery (Lin et al., 2026; Fan et al., 2026; Zhu et al., 2026; Wu et al., 2026; Gandhi et al., 2026; Dong et al., 2026). These components can be paired with many prompting policies. Our comparison isolates the policy choice itself: for a fixed synthesis and verification pipeline, which operation should the LLM apply to each seed?

This paper asks a concrete selection question: given a seed task and a current policy, which synthesis action should place the resulting environment near the learning frontier? Figure 1 shows the contrast. Envs-FORGE first estimates the seed pass rate from verifier rewards. It then scores six actions formed by a projection (increase, reduce, or diversify) and an evolution direction (in-depth or in-breadth). A per-seed MILP selects the action under feasibility constraints, and the chosen action conditions joint rewriting of the instruction, fixtures, oracle solution, tests, and Docker environment. The output is accepted only after gold verification.

1 Introduction

Terminal agents are becoming practical interfaces for software engineering, system administration,

^{*}Equal Contribution

[†]Corresponding Author

A task’s value for RL depends on its difficulty relative to the policy, not only on fluent wording. DART-Math allocates more response trials to difficult queries, and targeted tabular synthesis steers generators toward hard observations (Tong et al., 2024; Ferracci et al., 2024). Envs-FORGE applies the same frontier idea to executable environment construction: the decision variable is the transformation applied to a seed. Fixed recipes become restricted masks in the same action space. Few-shot preserves seed structure, Self-Instruct invents a related task, and Evol-Instruct fixes the direction of evolution. Envs-FORGE can choose both projection and direction, including a reduce-complexity projection that creates bridge tasks from seeds beyond the current policy.

On Qwen 3.5 35B trained with Group Relative Policy Optimization (GRPO), Envs-FORGE improves Pass@1 over Base from 40.0% to 49.2% on tb-core and from 23.0% to 29.4% on tb-2.0. It exceeds the strongest fixed-recipe baseline on each benchmark by 2.4 and 2.1 percentage points, respectively. All four synthesis methods operate at a broadly comparable scale: total token use ranges from 2.27M to 2.88M while each exports exactly 100 verified environments. We report this resource accounting alongside downstream performance.

Our main contributions are:

- We recast agent environment synthesis as reward-grounded action selection over verified seed tasks, with explicit increase, reduce, and diversify projections and in-depth/in-breadth evolution directions.
- We formulate per-task projection–direction selection as a mixed-integer program, represent the approximate semantics of fixed synthesis recipes as restricted action masks, and provide an indexed extension for optional soft skill coverage alongside synchronized artifact rewriting and gold verification.
- We evaluate against few-shot, Self-Instruct, and Evol-Instruct on tb-core, tb-2.0, and SWE-bench Verified, analyze tb-core across Qwen 3.5 model sizes, and report synthesis-resource accounting and qualitative environment traces.

2 Related Work

Terminal agents and benchmarks. Terminal interaction is a central setting for agentic mod-

els. SWE-agent and OpenHands instantiate tool-using execution loops, and Terminal-Bench and SWE-bench Verified evaluate containerized terminal tasks and repository repair (Yang et al., 2024; Wang et al., 2025; Merrill et al., 2026; OpenAI, 2024). These benchmarks provide executable evaluation targets, not a policy-conditioned rule for synthesizing new training environments.

Synthetic instruction and environment generation.

Self-Instruct, WizardLM, and AgentInstruct expand instruction data through self-generated examples, complexity evolution, or agentic flows (Wang et al., 2023; Xu et al., 2025; Mitra et al., 2024). Terminal-specific systems add components that change the synthesis substrate: CLI-Gym reverts healthy Docker runtimes to failure states; SkillSynth samples skill-graph paths; TermiGen constructs containers and injects trajectory errors; TerminalTraj builds Dockerized tasks from repositories; Endless Terminals procedurally generates verified tasks; and Agent-World discovers tasks from tool ecosystems and capability gaps (Lin et al., 2026; Fan et al., 2026; Zhu et al., 2026; Wu et al., 2026; Gandhi et al., 2026; Dong et al., 2026). These additions are compatible with our line of work as auxiliary runtime, perturbation, construction, or discovery components. Envs-FORGE addresses a different layer: the prompting policy that chooses how an LLM should transform each seed. We compare against few-shot, Self-Instruct, and Evol-Instruct as same-level fixed policies under a shared environment-generation and verification pipeline.

Difficulty-aware selection. Our formulation is related to difficulty-aware data selection and hardness-aware synthesis. DART-Math assigns more response-generation trials to difficult queries, and targeted tabular synthesis trains generators on observations identified as hard (Tong et al., 2024; Ferracci et al., 2024). Their decision concerns where to spend synthesis effort. Our decision concerns which transformation to apply to an executable seed. Envs-FORGE estimates the effect of each projection with a fixed transfer prior and selects the action closest to the learning frontier. Executable verification then checks whether the materialized bundle is runnable, gradeable, and internally consistent.

Why fixed synthesis misses the learning frontier

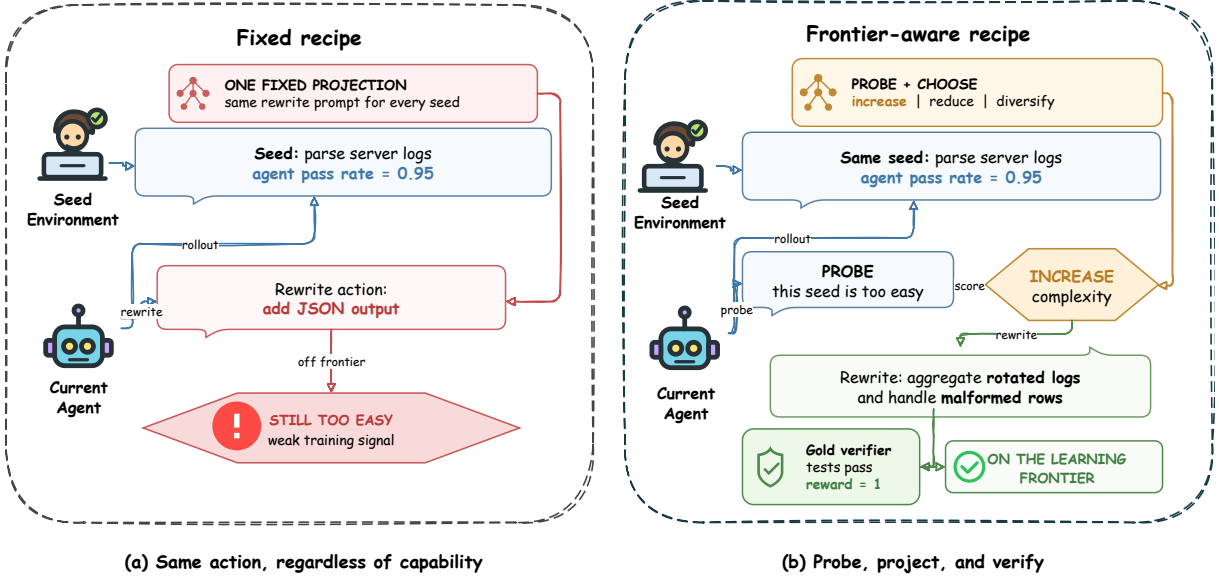


Figure 1: Motivation contrast. Fixed prompt recipes apply the same rewrite strategy to every seed, while Envs-FORGE estimates the seed’s pass rate first and then chooses among increase, reduce, and diversify projections to land near the learning frontier.

3 Method

3.1 Problem Setting and Overview

We study environment synthesis for terminal agents trained with reinforcement learning. A seed task is represented as $s_i = (I_i, D_i, S_i, T_i, E_i)$, where I_i is the instruction, D_i is the fixture and data bundle, S_i is an oracle solution, T_i is the test suite, and E_i is the executable environment. These components are coupled: changing the instruction without updating the solution or tests can produce a task that is either ungradeable or inconsistent with its stated objective.

Our method separates two decisions that fixed synthesis recipes merge into one prompt. First, it decides how a seed should move relative to the current agent. Second, it rewrites and verifies the complete environment under that decision. The projection set is

$$\mathcal{A} = \{\text{increase, reduce, diversify}\},$$

and the evolution-direction set is

$$\mathcal{D} = \{\text{in_depth, in_breadth}\}.$$

Each candidate action is $c_{i,a,d} = (s_i, a, d)$, with $a \in \mathcal{A}$ and $d \in \mathcal{D}$. Figure 2 summarizes the pipeline. Envs-FORGE estimates a frontier score for each candidate, solves one per-seed action-selection problem, conditions the synthesis prompt

on the selected action, and accepts only environments that pass static checks and gold verification. The reported Envs-FORGE run uses the MILP selector described below.

3.2 Frontier-Aware Candidate Scoring

The selector uses verifier rewards from the current policy. For seed i , let $r_{i,t} \in [0, 1]$ be the verifier reward from rollout t . We estimate the seed pass rate as

$$\hat{p}_i = \frac{1}{n_i} \sum_{t=1}^{n_i} r_{i,t},$$

where binary rewards yield the empirical success rate and partial rewards, when available, contribute proportionally. The estimate is computed before candidate construction and remains fixed during optimization. We approximate the post-projection pass rate of candidate $c_{i,a,d}$ as

$$\tilde{p}_{i,a,d} = \text{clip}(\hat{p}_i + \Delta_a \gamma_d, 0, 1), \quad (1)$$

where $\Delta_{\text{increase}} = -0.25$, $\Delta_{\text{reduce}} = 0.25$, and $\Delta_{\text{diversify}} = 0$ encode the intended difficulty movement. The factors $\gamma_{\text{in_depth}} = 1$ and $\gamma_{\text{in_breadth}} = 0.65$ attenuate breadth changes. These fixed transfer priors rank candidates before generation; they are not empirical claims that every materialized task shifts pass rate by the same amount.

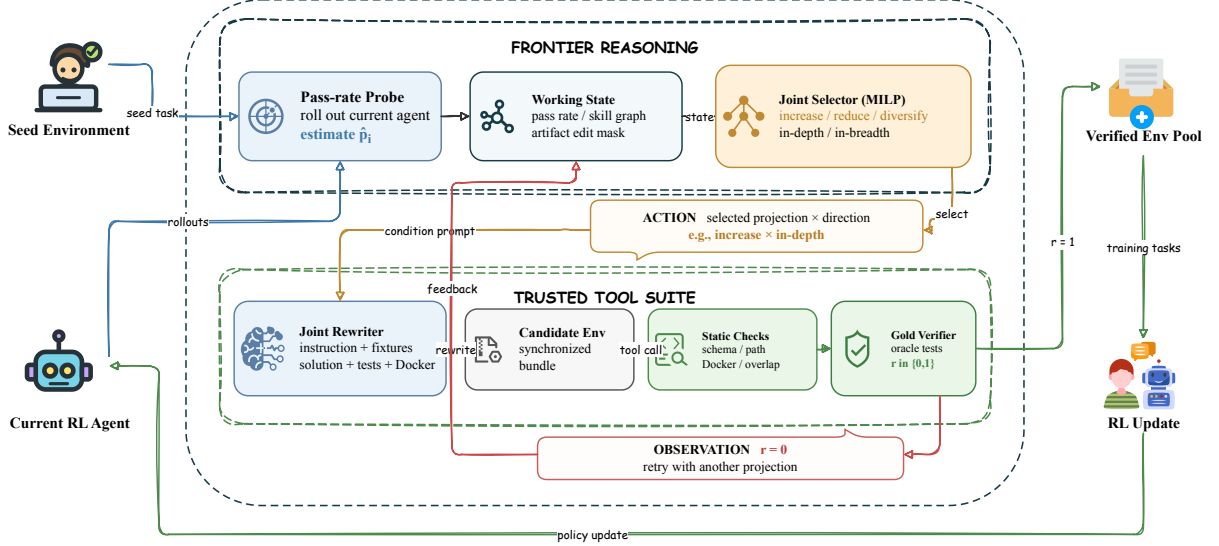


Figure 2: Envs-FORGE pipeline. Seed tasks are scored by a frontier-aware pass-rate estimator, and the MILP jointly chooses a projection and evolution direction before the synthesis model rewrites the full environment bundle. Only tasks that pass static checks and gold verification enter GRPO training.

We map the predicted pass rate to a learning-frontier score

$$F_{i,a,d} = \exp\left(-\frac{(\tilde{p}_{i,a,d} - \tau)^2}{2\sigma^2}\right), \quad (2)$$

$$\tau = 0.5, \quad \sigma = 0.2,$$

so that candidates near the target frontier receive higher scores. The three projections have distinct roles. *increase* adds concrete constraints, edge cases, stricter outputs, or larger fixtures for easy seeds. *reduce* removes secondary systems or creates a smaller bridge task for seeds beyond the current policy. *diversify* changes fixtures or neighboring requirements while preserving approximately the same difficulty and skill family.

Each candidate carries a skill-node set and a consistency contract. Let $R_{i,a,d}$ be required skill nodes and $O_{i,a,d}$ be optional nodes that the candidate may add or remove. Before solving, the metadata records prompt eligibility, length, split membership, and known overlap indicators. Schema, Docker, and executable-test checks are applied after materialization. This division lets the selector reason over frontier value, skill coverage, and feasibility signals while leaving semantic artifact consistency to executable verification.

3.3 Per-Task MILP Action Selection

The optimization unit is one seed task and its six candidate actions. For a given seed, the MILP selects one projection and one evolution direction; linked skill variables expose the subgraph induced

by that action. We write the equations in indexed form over seeds so independent per-task instances can be stacked for implementation and an optional portfolio mode can add shared skill-coverage targets. We introduce binary variables

$$x_{i,a,d} \in \{0, 1\}, \quad u_{i,a,d,v} \in \{0, 1\},$$

where $x_{i,a,d} = 1$ selects action (a, d) for seed i , and $u_{i,a,d,v} = 1$ activates skill node v for that candidate. For active skill-coverage targets m_v , we introduce continuous slack $0 \leq \xi_v \leq \bar{\xi}$; the reported experiments use $\bar{\xi} = 0.2$. Because the frontier scores and candidate metadata are precomputed before solving, the following objective is linear in the decision variables:

$$\begin{aligned} \max_{x,u,\xi} \quad & \sum_{i,a,d} x_{i,a,d} F_{i,a,d} - \varepsilon \sum_{i,a,d} \sum_{v \in O_{i,a,d}} u_{i,a,d,v} \\ & - \lambda \sum_v \xi_v, \end{aligned} \quad (3)$$

where $\varepsilon = 10^{-6}$ removes arbitrary activation of optional nodes and $\lambda = 0.25$ penalizes unmet coverage when coverage targets are active. The objective prioritizes frontier value, prefers compact skill realizations, and records bounded coverage shortfalls through ξ_v .

The core constraints enforce local action cardinality and action-skill consistency. Here, N is the number of selected seed-action instances in the indexed form. A per-task instance selects one action; the reported stacked export has $N = |\mathcal{S}|$.

The quantities m_v and ξ_v are instantiated for active skill-coverage targets, with the slack cap $\bar{\xi} = 0.2$ in the reported experiments. Candidates that fail eligibility checks, configured overlap filters, or the prompt-length budget are removed from the feasible set; equivalently, they can be represented by $x_{i,a,d} \leq v_{i,a,d}$, $x_{i,a,d} \leq 1 - \ell_{i,a,d}$, and $p_{i,a,d}x_{i,a,d} \leq P_{\max}$, where $v_{i,a,d}$, $\ell_{i,a,d}$, and $p_{i,a,d}$ denote eligibility, known overlap risk, and prompt length. Full artifact validity is checked after generation. The resulting linear constraints are

$$\sum_{a,d} x_{i,a,d} \leq 1 \quad \forall i, \quad (4)$$

$$\sum_{i,a,d} x_{i,a,d} = N, \quad (5)$$

$$u_{i,a,d,v} = x_{i,a,d} \quad \forall v \in R_{i,a,d}, \quad (6)$$

$$u_{i,a,d,v} \leq x_{i,a,d} \quad \forall v \in O_{i,a,d}, \quad (7)$$

$$u_{i,a,d,v} = 0 \quad \forall v \notin R_{i,a,d} \cup O_{i,a,d}, \quad (8)$$

$$\sum_{i,a,d} u_{i,a,d,v} + \xi_v \geq m_v \quad \forall v, \quad (9)$$

$$0 \leq \xi_v \leq \bar{\xi} \quad \forall v. \quad (10)$$

The formulation also makes the relationship to heuristic baselines explicit. Let $\rho_{b,a,d} \in \{0, 1\}$ be a mask for baseline b . Adding

$$x_{i,a,d} \leq \rho_{b,a,d} \quad \forall i, a, d \quad (11)$$

restricts the optimizer to a baseline-specific semantic view. Few-shot is approximated by (diversify, in_depth) because it preserves seed structure while changing concrete requirements; self-instruct is approximated by (diversify, in_breadth) because it invents a related task in the same domain; evol-depth fixes $d = \text{in_depth}$; and evol-breadth fixes $d = \text{in_breadth}$. These masks place fixed prompting policies in a shared action space. Envs-FORGE chooses both factors and adds the reduce projection for bridge-task construction.

Implementation boundary. The reported experiment instantiates the selector in its intended per-seed mode. Each of the 100 seeds solves a six-action MILP and selects one action; the trace serializes these decisions in the indexed form with $N = 100 = |S|$ and active coverage slack capped at $\bar{\xi} = 0.2$. Shared coverage targets and smaller portfolio sizes remain optional modes for curricula with explicit skill quotas, and Appendix B reports

the solver trace. The main evaluation measures the complete per-seed selector, synchronized materialization, and verification pipeline under the same 100-environment export size as the fixed-policy baselines.

3.4 Verified Environment Synthesis

After selection, a synthesis model receives the projection, evolution direction, preserved skill sub-graph, and artifact-consistency contract. It rewrites the instruction, fixture bundle, oracle solution, tests, and Docker environment jointly. The prompt forbids instruction-only edits and hidden test requirements, and it requires the verifier output to be materialized in the expected reward file. The high-level action becomes a complete Terminal-Bench-style task.

We then apply schema, path-safety, length, Docker, test, and overlap checks. An environment enters the training pool only if its oracle solution obtains reward 1 under the generated tests. Gold verification is part of the data definition: generation can explore many candidates, but training uses only synchronized, runnable, and gradeable environments.

Algorithm 1 summarizes the complete selection, joint materialization, and verification procedure.

Implementation note. The reported run applies one local MILP formulation per seed; for audit, the 100 local decisions are serialized in the indexed model in Eqs. 3–10. Active coverage slack is capped at $\bar{\xi} = 0.2$. A portfolio deployment may instead use smaller export targets or tighter coverage slack.

3.5 GRPO Training and Evaluation

Accepted environments are converted to a common Terminal-Bench task format and used for GRPO training (Shao et al., 2024). The main comparison trains Qwen 3.5 35B (Yang et al., 2025); the model-size analysis additionally evaluates 4B, 9B, and 27B settings on tb-core. vLLM generates rollouts, and task tests provide rewards without a separate learned reward model. The training and evaluation protocols are fixed across synthesis sources within each comparison. Synthesis-resource use is reported separately and remains within the same overall scale across methods.

Algorithm 1 Frontier-aware environment synthesis

Require: Seed tasks $\mathcal{S}$ and current policy π

Ensure: Verified environment batch $\mathcal{B}$

```
1:  $\mathcal{B} \leftarrow \emptyset$ 
2: for each seed  $s_i \in \mathcal{S}$  do
3:   Estimate  $\hat{p}_i$  from policy rollouts
4:   for each  $(a, d) \in \mathcal{A} \times \mathcal{D}$  do
5:     Derive  $R_{i,a,d}$ ,  $O_{i,a,d}$ , and the artifact
       contract
6:     Compute  $\tilde{p}_{i,a,d}$  and  $F_{i,a,d}$ 
7:     Remove candidates that fail pre-solve
       eligibility checks
8:   end for
9:   Solve the six-action per-task MILP for seed
        $s_i$  and select  $(a_i, d_i)$ 
10:  Jointly materialize environment bundle
        $\tilde{s}_{i,a_i,d_i}$ 
11:  Check its schema, paths, prompt length,
       overlap, and container isolation
12:  Build  $\tilde{s}_{i,a_i,d_i}$ ; run its oracle solution and
       generated tests
13:  if the verifier returns reward 1 then
14:     $\mathcal{B} \leftarrow \mathcal{B} \cup \{\tilde{s}_{i,a_i,d_i}\}$ 
15:  end if
16: end for
17: return  $\mathcal{B}$ 
```

4 Experiments

4.1 Setup

We organize the evaluation around two questions: (1) does frontier-aware environment synthesis improve downstream agent performance over fixed prompting recipes, and (2) what synthesis work is needed to reach the shared 100-environment export size?

Training sources and verification. We compare four synthetic training sources: few-shot prompting, Self-Instruct, Evol-Instruct with both in-depth and in-breadth evolution, and Envs-FORGE with MILP selection. Each run continues generation, repair, and verification until exactly 100 environment bundles are accepted and exported as the downstream RL training source. All synthesized conditions contribute the same number of verified training tasks. Source records, materialized task directories, attempts, and tokens measure the synthesis-stage work required to reach that endpoint. The Base row uses no synthesized training data.

Training and evaluation. The main comparison uses Qwen 3.5 35B (Yang et al., 2025) trained with GRPO (Shao et al., 2024); the model ablation additionally evaluates 4B, 9B, 27B, and 35B settings. Rollouts use vLLM and test-based rewards without a separate learned reward model. We evaluate Pass@1 on tb-core and tb-2.0 with a common protocol, add SWE-bench Verified (OpenAI, 2024) in the benchmark ablation, and use tb-core for the model-size analysis. We report benchmark scores and percentage-point differences under this shared evaluation protocol.

Baselines and resource accounting. Few-shot and Self-Instruct instantiate fixed prompting recipes; Evol-Instruct combines its in-depth and in-breadth variants. These controls isolate prompting policy while holding artifact generation and verification fixed. The auxiliary components in Section 2 can be paired with any policy, so we keep them outside this controlled comparison. Envs-FORGE uses the MILP selector in Section 3.3. We report records, accepted outputs, task directories, attempts, and token totals. The methods materialize 194–210 task directories and use 2.27M–2.88M tokens to export 100 verified environments each; rejected records and intermediate artifacts do not enter RL training.

4.2 Main Results

Envs-FORGE obtains the highest Pass@1 on both benchmarks. Table 1 shows that the Base model obtains 40.0% on tb-core and 23.0% on tb-2.0. Few-shot improves these scores to 43.2% and 24.1%, Self-Instruct to 45.6% and 27.3%, and Evol-Instruct to 46.8% and 25.6%. Envs-FORGE reaches 49.2% on tb-core and 29.4% on tb-2.0, giving gains of 9.2 and 6.4 percentage points over Base. Against the strongest fixed-recipe baseline for each benchmark, the margins are 2.4 points over Evol-Instruct on tb-core and 2.1 points over Self-Instruct on tb-2.0. Figure 3(a) visualizes these scores.

Interpretation. Across these runs, frontier-aware selection with verified environment synthesis yields the highest Pass@1 on both benchmarks. The fixed policies help over Base, but their single rewrite direction leaves some seeds too easy, too hard, or too close to the original task. Envs-FORGE instead chooses the seed-level action before generation.

Table 1: Training-data synthesis statistics and downstream Pass@1. Each synthesis run exports exactly 100 gold-verified environment bundles (the *Accepted* column) to downstream RL, with task directories and token use remaining in the same broad range across methods. Records, task directories, attempts, and token counts are synthesis-stage totals used to reach that fixed endpoint; they do not increase the downstream training-set cardinality. The Base row uses no synthesized training data.

Training data / model	Method	Records	Accepted	Task dirs	Attempt sum	Prompt tokens	Final completion tokens	Total synthesis tokens	tb-core Pass@1	tb-2.0 Pass@1
Base	—	—	—	—	—	—	—	—	40.0%	23.0%
few-shot	few_shot	121	100	210	226	1,835,473	607,592	2,443,065	43.2%	24.1%
Self-Instruct	self_instruct	113	100	194	190	1,745,028	527,247	2,272,275	45.6%	27.3%
	evol_instruct_in_depth +									
Evol-Instruct	evol_instruct_in_breadth	112	100	195	207	1,974,723	543,023	2,517,746	46.8%	25.6%
Envs-FORGE	MILP	120	100	203	291	1,862,146	618,910	2,881,056	49.2%	29.4%

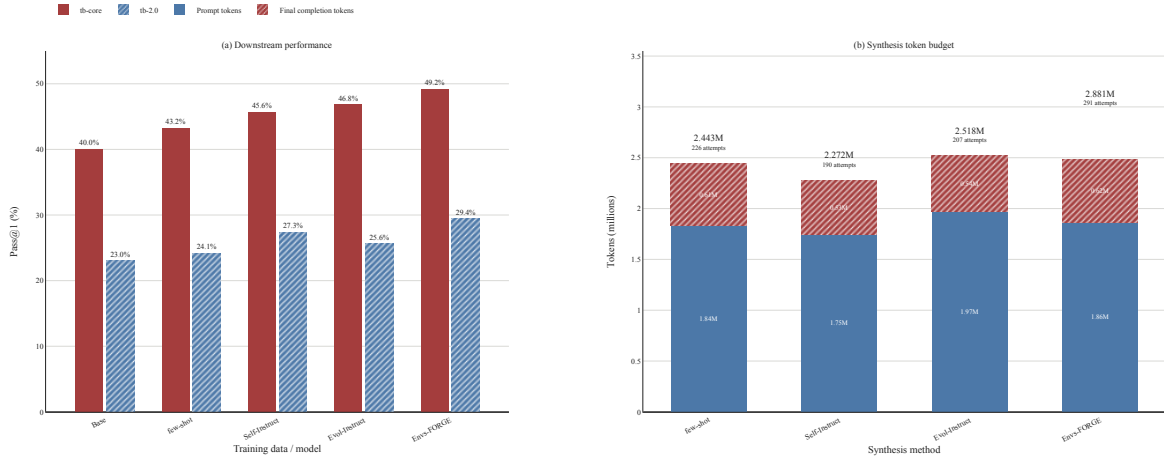


Figure 3: Downstream performance and synthesis cost. (a) Pass@1 on tb-core and tb-2.0; the Envs-FORGE run is highest on both benchmarks. (b) Synthesis-stage prompt and final-completion token totals for the four methods; labels above bars show total tokens and attempt counts. Totals occupy a comparable 2.27M–2.88M range, and each method exports exactly 100 gold-verified environments to RL. The bars measure the cost of reaching a fixed training-set size, not extra downstream training examples.

4.3 Ablation Studies

We next test whether the observed advantage is specific to the two Terminal-Bench-style scores or to the largest model setting.

Benchmark coverage. The benchmark ablation adds SWE-bench Verified to tb-core and tb-2.0. As summarized in Table 2(a) and Figure 4(a), Envs-FORGE is the highest-scoring condition on all three benchmarks: 49.2% on tb-core, 29.4% on tb-2.0, and 77.1% on SWE-bench Verified. Relative to Base, the gains are +9.2, +6.4, and +3.7 percentage points; relative to the strongest fixed-recipe baseline on each benchmark, the margins are +2.4, +2.1, and +1.3 points.

Model coverage. The tb-core model ablation in Table 2(b) and Figure 4(b) reports gains across all four Qwen 3.5 sizes. Envs-FORGE improves over

Base by 6.8 points at 4B, 7.2 at 9B, 8.1 at 27B, and 9.2 at 35B.

4.4 Synthesis Cost Analysis

Synthesis costs remain broadly comparable. Panel (b) of Figure 3 shows total synthesis use of 2.272M–2.881M tokens across the four methods, and Table 1 shows 194–210 materialized task directories. Table 3 places tokens per accepted environment in a 22,723–28,811 range and attempts per accepted environment in a 1.90–2.91 range. These aggregates are the same order of magnitude, and each method exports exactly 100 verified environments to RL. The execution profiles differ inside this shared scale: Envs-FORGE uses 291 shorter attempts and has the lowest average tokens per attempt (9,901), while the fixed recipes use 190–226 attempts averaging 10,810–12,163 tokens. Attempts and intermediate directories characterize

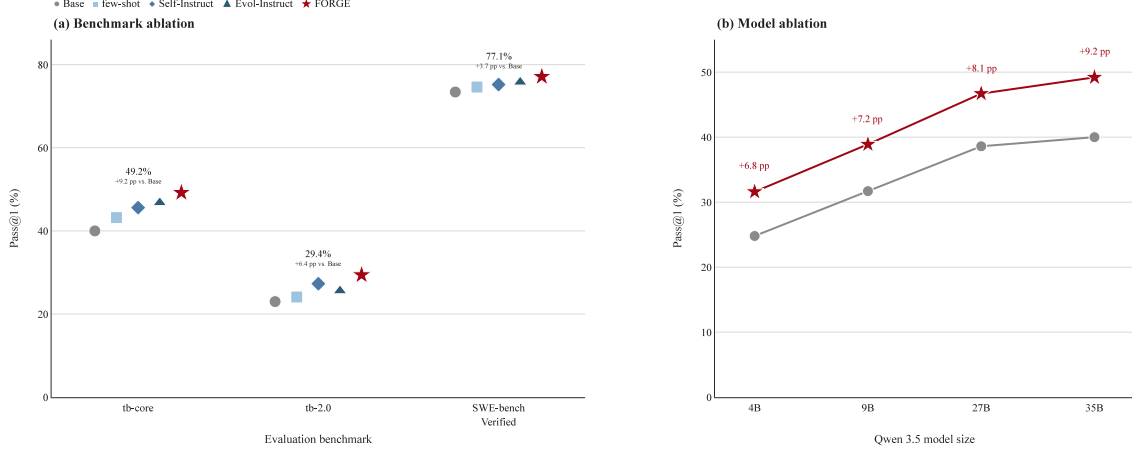


Figure 4: Benchmark and model ablations. (a) Pass@1 for the five training conditions across tb-core, tb-2.0, and SWE-bench Verified; labels show FORGE values and gains over Base. Marker shape redundantly encodes the training condition. (b) Base and FORGE tb-core Pass@1 across Qwen 3.5 model sizes; labels show the paired gain at each size.

Table 2: Benchmark and model ablations. In (a), bold marks the best training condition for each benchmark. Panel (b) reports tb-core, where Δ is FORGE minus Base in percentage points.

(a) Benchmark ablation						(b) Model ablation			
Benchmark	Base	few-shot	Self-Inst.	Evol-Inst.	FORGE	Model	Base	FORGE	Δ
tb-core	40.0%	43.2%	45.6%	46.8%	49.2%	Qwen 3.5 4B	24.8%	31.6%	+6.8
tb-2.0	23.0%	24.1%	27.3%	25.6%	29.4%	Qwen 3.5 9B	31.7%	38.9%	+7.2
SWE-bench Verified	73.4%	74.6%	75.2%	75.8%	77.1%	Qwen 3.5 27B	38.6%	46.7%	+8.1
						Qwen 3.5 35B	40.0%	49.2%	+9.2

Table 3: Derived synthesis-cost statistics. Each synthesis run accepts 100 outputs, and all normalized costs remain within the same overall scale. Values are computed from the aggregate counts in Table 1.

Training data	Attempts / accepted	Tokens / accepted	Tokens / attempt
few-shot	2.26	24,431	10,810
Self-Instruct	1.90	22,723	11,959
Evol-Instruct	2.07	25,177	12,163
Envs-FORGE	2.91	28,811	9,901

the synthesis route to the fixed acceptance endpoint; they do not enlarge the downstream training set.

4.5 Evaluation Scope

The evaluation compares complete prompting policies at a fixed 100-task export and a comparable synthesis scale. Expanded component studies can isolate portfolio mode, solver-off selection, transfer-prior sensitivity, and Evol-Instruct’s depth and breadth variants.

5 Conclusion

We present Envs-FORGE, a frontier-aware prompting policy for terminal-agent environment synthesis. The core idea is to choose a per-seed synthesis action from verifier-derived difficulty estimates, then use that action to condition synchronized and gold-verified environment rewriting. On Qwen 3.5 35B, Envs-FORGE obtains the highest Pass@1 among the evaluated methods on tb-core, tb-2.0, and SWE-bench Verified, improving over Base by 9.2, 6.4, and 3.7 percentage points. The results support a simple lesson: environment synthesis for agent RL should decide how each seed should move relative to the current policy before asking an LLM to rewrite it.

Limitations

This paper evaluates the per-seed MILP mode of Envs-FORGE, which is the setting used for the reported environment-synthesis run with active coverage slack capped at $\bar{\xi} = 0.2$. The indexed formulation also supports portfolio-level skill quotas with different export targets or slack budgets, but those settings are outside the present comparison. The study compares complete prompting policies at a fixed 100-environment export size; finer component studies can test solver-off selection, transfer-prior sensitivity, and the depth and breadth variants of Evol-Instruct separately. Scaling the seed pool and model family is the natural next step.

Ethical Considerations

This work studies environment synthesis for terminal agents. The experiments use benchmark tasks and existing task and experiment records; no human-subject data are collected. More effective environment synthesis can reduce repeated operational failures and improve the reliability of software-engineering agents, but it can also increase agent capability and persistence in command-line settings. Envs-FORGE records solver decisions and verification outcomes, applies overlap and artifact-consistency checks, and admits an environment only after gold verification so that synthesis decisions remain inspectable. These safeguards do not remove risks inherited from the base model, execution system, tools, or benchmark data. Deployment should include task-appropriate permissions, sandboxing, logging, and human oversight.

Information About Use of AI Assistants

In preparing this manuscript, the authors used AI-assisted tools, including large language models such as GPT-5 and DeepSeek-V4, for text refinement. Their use was limited to proofreading, grammatical correction, and polishing linguistic expressions to improve clarity and readability. The authors are responsible for the final content, technical claims, citations, experimental results, and verification.

References

- Guanting Dong, Junting Lu, Junjie Huang, Wanjun Zhong, Longxiang Liu, Shijue Huang, Zhenyu Li, Yang Zhao, Xiaoshuai Song, Xiaoxi Li, Jiajie Jin, Yutao Zhu, Hanbin Wang, Fangyu Lei, Qinyu Luo, Mingyang Chen, Zehui Chen, Jiazhan Feng, Ji-Rong Wen, and Zhicheng Dou. 2026. [Agent-world: Scaling real-world environment synthesis for evolving general agent intelligence](#). *Preprint*, arXiv:2604.18292.
- Zhiyuan Fan, Tinghao Yu, Yuanjun Cai, Jiangtao Guan, Yun Yang, Dingxin Hu, Jiang Zhou, Xing Wu, Zhuo Han, Feng Zhang, and Lilin Wang. 2026. [Toward scalable terminal task synthesis via skill graphs](#). *Preprint*, arXiv:2604.25727.
- Tommaso Ferracci, Leonie Tabea Goldmann, Anton Hinel, and Francesco Sanna Passino. 2024. Targeted synthetic data generation for tabular data via hardness characterization. *arXiv preprint arXiv:2410.00759*.
- Kanishk Gandhi, Shivam Garg, Noah D. Goodman, and Dimitris Papailiopoulos. 2026. [Endless terminals: Scaling rl environments for terminal agents](#). *Preprint*, arXiv:2601.16443.
- Yusong Lin, Haiyang Wang, Shuzhe Wu, Lue Fan, Feiyang Pan, Sanyuan Zhao, and Dandan Tu. 2026. [Cli-gym: Scalable cli task generation via agentic environment inversion](#). *Preprint*, arXiv:2602.10999.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jernia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, and 66 others. 2026. [Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces](#). *Preprint*, arXiv:2601.11868.
- Arindam Mitra, Luciano Del Corro, Guoqing Zheng, Shweti Mahajan, Dany Rouhana, Andres Coda, Yadong Lu, Wei ge Chen, Olga Vrousgos, Corby Rosset, Fillipe Silva, Hamed Khanpour, Yash Lara, and Ahmed Awadallah. 2024. [Agentinstruct: Toward generative teaching with agentic flows](#). *Preprint*, arXiv:2407.03502.
- OpenAI. 2024. Introducing SWE-bench verified. <https://openai.com/index/introducing-swe-bench-verified/>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, and 1 others. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Yuxuan Tong, Xiwen Zhang, Rui Wang, Ruidong Wu, and Junxian He. 2024. Dart-math: Difficulty-aware rejection tuning for mathematical problem-solving.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, and 5 others. 2025. [Openhands: An open platform for ai software developers as generalist agents](#). *Preprint*, arXiv:2407.16741.
- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. [Self-instruct: Aligning language models with self-generated instructions](#). *Preprint*, arXiv:2212.10560.
- Siwei Wu, Yizhi Li, Yuyang Song, Wei Zhang, Yang Wang, Riza Batista-Navarro, Xian Yang, Mingjie Tang, Bryan Dai, Jian Yang, and Chenghua Lin. 2026. [Large-scale terminal agentic trajectory generation from dockerized environments](#). *Preprint*, arXiv:2602.01244.
- Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, Qingwei Lin, and Daxin Jiang. 2025. [Wizardlm: Empowering large pre-trained language models to follow complex instructions](#). *Preprint*, arXiv:2304.12244.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. [Qwen3 technical report](#). *Preprint*, arXiv:2505.09388.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. [Swe-agent: Agent-computer interfaces enable automated software engineering](#). *Preprint*, arXiv:2405.15793.
- Kaijie Zhu, Yuzhou Nie, Yijiang Li, Yiming Huang, Jialian Wu, Jiang Liu, Ximeng Sun, Zhenfei Yin, Lun Wang, Zicheng Liu, Emad Barsoum, William Yang Wang, and Wenbo Guo. 2026. [Termigen: High-fidelity environment and robust trajectory synthesis for terminal agents](#). *Preprint*, arXiv:2602.07274.

Appendix Overview

The appendix is organized as follows.

- Appendix A expands the optimization and reproducibility details, including action semantics, baseline restrictions, the solver-generation boundary, artifact-consistency checks, training and evaluation settings, and five qualitative frontier cases.
- Appendix B describes MILP model assembly, SCIP’s branch-and-cut procedure, the recorded synthesis solve, solution decoding, fallback behavior, and the boundary of the solver’s correctness guarantee.
- Appendix C reproduces the core baseline and Envs-FORGE prompt clauses, including strategy injection, repair, the solver payload, projection instructions, artifact edit masks, the skill-subgraph contract, and original-to-synthesized task excerpts.
- Appendix D reports the compute resources used for synthesis, training, and evaluation.

A Optimization and Reproducibility Details

This appendix expands the action semantics, solver boundary, environment-verification protocol, and qualitative cases summarized in the main paper.

A.1 Action Semantics and Baseline Restrictions

The projection and evolution direction answer different questions. The projection specifies how difficulty should move relative to the current policy: increase an easy seed, reduce a hard seed into a bridge task, or diversify a seed already near the frontier. The direction specifies whether the rewrite follows the same skill chain (*in_depth*) or moves to a neighboring skill or task type (*in_breadth*). Table 4 records the operational contracts used to instantiate these choices.

The restricted policies in the lower half of Table 4 make the baseline-inclusion claim precise. If $\mathcal{X}_{\text{FORGE}}$ denotes the feasible set defined by Eqs. 4–9, then baseline b induces

$$\mathcal{X}_b = \{x \in \mathcal{X}_{\text{FORGE}} : x_{i,a,d} \leq \rho_{b,a,d} \forall i, a, d\}. \quad (12)$$

This gives a unified optimization view of the prompting policies. The underlying prompts remain distinct: few-shot is structurally narrower

than generic diversification, Self-Instruct permits freer same-domain invention, and the two Evolve-Instruct variants fix an evolution direction. Envs-FORGE selects both the projection and direction for each seed from the full feasible region using policy-relative frontier scores; optional portfolio mode adds shared skill-coverage targets across those local instances.

A.2 Candidate Construction and Solver Boundary

Each seed induces six candidate action variants and one local MILP instance. Before solving, the pipeline estimates the seed pass rate, applies the fixed projection–direction transfer heuristic in Eq. 1, computes the frontier utility in Eq. 2, and derives required and optional skill nodes. Candidate prompt length, split membership, and known overlap or validity indicators are also treated as constants. The local MILP jointly selects the projection and evolution direction; the indexed equations additionally expose skill activation and optional portfolio coverage. Table 5 lists the skill-node vocabulary, and Table 6 summarizes the complete notation.

The artifact edit mask is outside the solver. It is deterministically derived from the selected action and passed to generation as a consistency contract. Joint materialization and executable verification enforce semantic dependencies among instructions, fixtures, solutions, tests, and containers; the MILP handles the discrete action choice. Likewise, $\rho_{b,a,d}$ is an analysis-time constant for defining restricted policies. It is not a free variable in the default Envs-FORGE solve. Failed verification attempts are excluded from the training set; generation or repair can continue only within the synthesis budget for that run.

Section B details model assembly, branch-and-cut solution, and solver-trace validation. Section C reproduces the core prompt clauses that connect the solver decision to generation, including the projection instruction, artifact edit mask, and skill-subgraph contract. It also records the shared baseline template and the strategy instruction that distinguishes each fixed recipe.

A.3 Artifact-Consistency Contract

A materialized sample is a complete executable environment, not an instruction-only rewrite. The contract requires synchronized versions of (i) the natural-language instruction, (ii) input fixtures and

Table 4: Operational semantics of the Envs-FORGE action space and its restricted baseline views. The baseline correspondences describe approximate prompt semantics, not exact equivalence between generation procedures.

Projection	Agent-relative regime	Permitted movement	Invariants
increase	Seed is too easy	Add one or two constraints, edge cases, larger fixtures, or stricter deterministic outputs.	Preserve the core skill subgraph; update the solution and tests to cover every added requirement.
reduce	Seed is too hard	Remove secondary systems or replace brittle infrastructure with a smaller bridge task.	Preserve the target skill path and its essential artifacts; simplification must not collapse the task into an unrelated problem.
diversify	Seed is near the frontier	Change fixtures, neighboring requirements, or scenario while keeping similar difficulty.	Avoid unrelated breadth jumps; synchronize expected outputs and verifier logic with the new fixture.

Restricted policy	Mask in the per-task action space	Interpretation
few-shot	(a, d) (diversify, in_depth)	= Approximate semantics: generate a close seed-conditioned variant while preserving the seed’s core structure.
Self-Instruct	(a, d) (diversify, in_breadth)	= Approximate semantics: invent a related task in the same domain with synchronized artifacts.
Evol-Instruct depth	$d = \text{in_depth}$	Approximate semantics: fix the evolution direction to deeper constraints or reasoning steps.
Evol-Instruct breadth	$d = \text{in_breadth}$	Approximate semantics: fix the evolution direction to a neighboring skill or task type.

Table 5: Skill-node taxonomy used for candidate metadata. These nodes define the required set $R_{i,a,d}$ and optional addable or removable set $O_{i,a,d}$ attached to each action candidate; they are metadata tags for the action contract rather than separate generation artifacts.

Category	Skill nodes
Domain	system_admin, data_processing, testing, security, ml_ops, game_reasoning
Tool	bash, python, systemd, docker, sql, node
Artifact	logs, csv, json, jsonl, source_code, config
Operation	merge, parse, repair, configure, generate_tests, rank, validate
Constraint	concurrency, scheduling, atomicity
FORGE-added nodes	edge_cases, structured_output, deterministic_sorting, alternate_fixture, neighboring_constraint, file_level_simulation, deterministic_fixture

data, (iii) the oracle solution, (iv) executable tests and reward logic, and (v) the container environment. Added requirements must be visible in the instruction and tested by the verifier; removed requirements must disappear from both the oracle and tests. The container may expose task fixtures but may not package the oracle solution or hidden tests into the agent-visible environment.

Verification has two layers. Static checks reject malformed schemas, unsafe paths, missing required files, inconsistent reward output, excessive prompt length, and overlap with held-out evaluation tasks. Executable checks then build the isolated environment, run the oracle solution, and invoke the generated tests. A task is accepted only when the verifier emits reward 1. This gold reward certifies internal task consistency; it is distinct from the downstream policy reward used to

estimate learning difficulty.

A.4 Preprocessing, Training, and Evaluation Protocol

Normalization and filtering. Accepted bundles are normalized into a common Terminal-Bench-style schema before conversion to training records. Each method contributes exactly 100 accepted bundles to the downstream RL source; failed candidates, repair attempts, and intermediate materializations are excluded from that source. The conversion verifies the Docker specification and test entry point, standardizes task metadata, and drops invalid filesystem artifacts. Prompt length is computed with the target tokenizer, the actual agent system prompt, and the full chat template. The experimental protocol uses a 4096-token threshold for the smaller settings and an 8192-token threshold

Table 6: MILP notation and implementation boundary. Candidate scores and metadata are constants when the solver is invoked.

Symbol	Meaning	Symbol	Meaning
s_i	Seed environment bundle	$\hat{p}_i$	Current policy pass-rate estimate on seed i
a, d	Projection and evolution direction	$\tilde{p}_{i,a,d}$	Predicted pass rate after applying (a, d)
$F_{i,a,d}$	Frontier utility of a candidate	$x_{i,a,d}$	Binary candidate-selection variable
$R_{i,a,d}$	Required skill nodes	$O_{i,a,d}$	Optional addable or removable skill nodes
$u_{i,a,d,v}$	Binary activation of skill node v	ξ_v	Continuous shortfall for target coverage m_v ; capped by $\xi = 0.2$ in reported experiments
N	Selected seed–action count in indexed portfolio form; one per local task	$\rho_{b,a,d}$	Constant action mask for restricted baseline b

for the 35B setting; filtering statistics are recorded before training, and prompts are not silently truncated.

Preflight validation. Before a training run, the pipeline loads every normalized task, rechecks the configured train/evaluation split and overlap filters, builds representative containers, and executes the same oracle-plus-test path used during synthesis. These checks are completed before model workers are launched so that container or verifier failures cannot consume rollout budget. Seed and evaluation splits remain separate throughout selection and downstream evaluation.

Optimization and rollout configuration. The reported 35B policy is trained with GRPO using test-derived rewards and no learned reward model. Training uses FSDP2 with parameter and activation offload, gradient checkpointing, and bfloat16 computation. vLLM generates asynchronous rollouts with eight samples per prompt, temperature 1.0, top- p 0.9, a 1024-token response cap, and at most 50 agent steps; the key-value cache uses FP8. The 35B run uses two H800 80 GB GPUs for training, while Pass@1 evaluation uses one H800 80 GB GPU. All methods use the same downstream training and evaluation protocol; only their synthesized training sources differ.

Evaluation scope. The evaluation compares the complete frontier-aware and verifier-backed pipeline under the shared downstream protocol. Each method exports 100 verified environments, and its synthesis totals remain within the same broad operational range as the other methods. A solver-off comparison over an identical candidate pool would add a component-level analysis of selector behavior.

A.5 Qualitative Frontier Cases

Figure 5 traces five verified environments from seed contracts through policy-relative action selection, synchronized materialization, and executable verification. For each case, the figure reports the estimated seed pass rate, the projected rate and frontier score under the selected action, the concrete contract changes, and the preserved skill path. The red, blue, and green panels cover complexity increase, complexity reduction, and frontier diversification. Original-to-synthesized instruction excerpts for all five cases appear in Section C.3.

Case I: frontier-guided hardening of a near-frontier Bash task. The Bash seed already exercises recursive traversal, concurrency, and idempotence, and its estimated pass rate of 0.747 places it slightly on the easy side of the target frontier. The selected depth increase projects the pass rate to 0.497 and receives a frontier score of 0.9999. The materialized environment keeps the Bash execution path but replaces loosely structured text artifacts with deterministic JSON summaries, including nested traversal, unusual filenames, C-locale ordering, file locking, and atomic writes. These additions create observable failure modes in the generated tests instead of merely lengthening the instruction.

Case II: verifier-facing hardening of a saturated merger. The original multi-format merger has an estimated pass rate of 1.0, so another close paraphrase would provide little learning signal. Its selected depth increase retains CSV, JSONL, and JSON parsing while projecting the rate to 0.750. The new contract defines the exact union key, treatment of missing fields, standards-compliant quoting, header schema, and deterministic row order. A superficially plausible merge is insufficient: the agent must satisfy edge-case and artifact-level requirements that the oracle and tests enforce jointly.

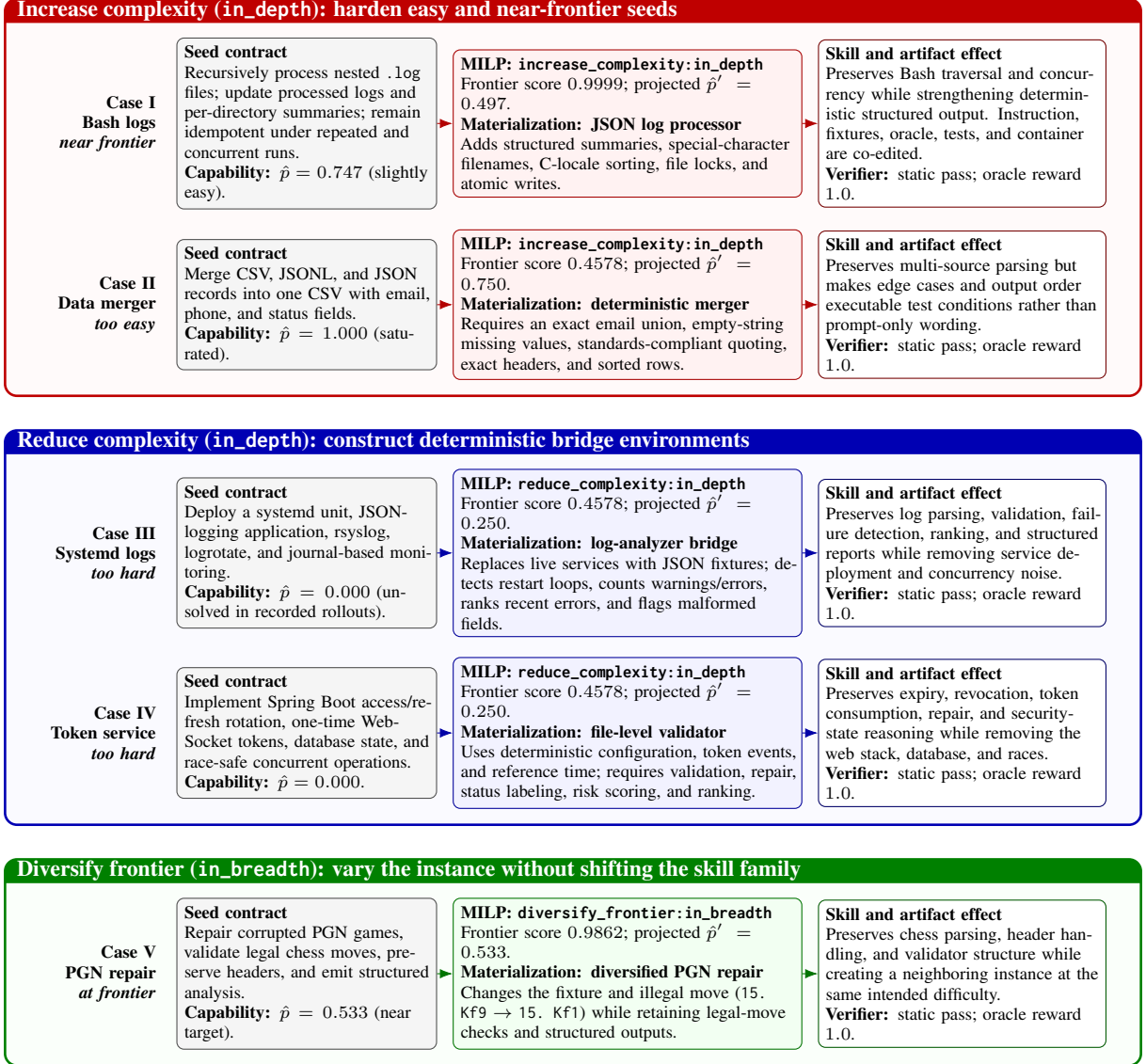


Figure 5: Detailed agentic case study of policy-relative environment synthesis. Each row follows a seed contract through the MILP decision and synchronized materialization to its preserved learning signal and verification result. The capability estimate $\hat{p}$, projected rate $\hat{p}'$, and frontier score motivate each action: red panels denote complexity increases for easy seeds, blue panels denote deterministic reductions for seeds unsolved in the recorded rollouts, and the green panel denotes frontier-preserving diversification. All five materializations co-edit the required task artifacts, pass static validation, and obtain oracle reward 1.0. These traces illustrate selection and consistency semantics; downstream effects are evaluated by the benchmark results.

Case III: a deterministic bridge from system deployment to log reasoning. The systemd seed combines application code with service installation, rsyslog, log rotation, and journal monitoring, yielding an estimated pass rate of 0.0. Envs-FORGE selects reduction because the seed was unsolved in the recorded rollouts. The bridge replaces live services with deterministic JSON fixtures but preserves restart-loop detection, severity counting, malformed-record validation, recent-error ranking, and structured reporting. The projected rate rises to 0.250, making the environment easier without discarding the log-analysis skill chain that motivated the seed.

Case IV: isolating security-state reasoning from a full web stack. The token-service seed is also estimated at 0.0 because it couples refresh rotation and one-time WebSocket tokens with framework setup, persistent database state, and race-safe concurrency. The reduced environment fixes configuration, event records, and reference time, then asks the agent to validate, repair, label, score, and rank token states. This removes Java, web-server, database, and concurrency overhead while retaining expiry, revocation, consumption, repair, and risk reasoning. The result is a projected 0.250 bridge task whose reward focuses on the intended security transitions rather than deployment failures.

Case V: frontier-preserving chess diversification. The PGN seed is already near the frontier at 0.533, so increasing or reducing its intended difficulty is unnecessary. The selected breadth action retains chess parsing, header preservation, legal-move validation, and structured repair, but changes the corrupted game and concrete illegal move from 15. Kf9 to its legal repair 15. Kf1. Its projected pass rate remains 0.533 and the frontier score is 0.9862. This example separates useful instance diversity from uncontrolled task drift: the fixture changes, whereas the skill family and verifier structure remain stable.

Cross-case interpretation. Together, the cases show why a mixed seed pool needs more than one fixed evolution direction. The same action space calls for hardening when a seed is saturated, reduction when infrastructure masks the target reasoning skill, and breadth diversification when the seed is already near the frontier. In every case, the instruction, fixtures, oracle, tests, and environment are updated under one artifact contract, and the ora-

cle obtains reward 1 after static validation. These traces establish internal executability and synchronization; the downstream performance evidence is reported in Section 4.2.

B MILP Solution Procedure

This section details how the environment-synthesis policy in Section 3.3 is instantiated and solved. Envs-FORGE uses PySCIPOpt as the modeling interface and SCIP as the solver. The optimization unit is a synthesis action, not a generated token, artifact, or training step. For each seed, the candidate set combines one of three difficulty projections with an in-depth or in-breadth evolution direction. Each candidate specifies a complete pre-materialization contract: the intended difficulty movement, evolution direction, frontier score, and required or optional skill nodes. The MILP selects which contract is materialized into an executable environment.

Model assembly. For every eligible candidate $c = (i, a, d)$, the implementation creates a binary action variable x_c . In the core per-task instance, the local cardinality constraint selects exactly one of the six candidates for the current seed. The indexed equations also include a cardinality target N for stacking local instances and allow shared skill-coverage constraints. Binary variables $u_{c,v}$ expose the skill subgraph induced by a selected action, and continuous variables ξ_v record shortfall against configured coverage targets. Active slack is bounded by $\bar{\xi} = 0.2$ in the reported experiments. All frontier scores, eligibility indicators, and skill sets are computed before model construction, so Eqs. 3–10 remain linear. Candidates that fail deterministic eligibility, overlap, or prompt-length checks are removed before optimization.

The formulation requires no big- M constants. A required skill satisfies $u_{c,v} = x_c$, a forbidden skill is fixed to zero, and an optional skill satisfies $u_{c,v} \leq x_c$ with a small activation penalty. The implementation allocates these linking variables over a uniform candidate-skill index to keep the decoded solver payload auditable. The primary combinatorial choice is which of the six synthesis-action variables to activate for each seed. Skill-variable values are fixed or bounded by x_c , and SCIP presolve can remove much of this deterministic structure. Soft coverage terms use ξ_v to tolerate at most 0.2 shortfall on active targets.

Branch-and-cut search. For this linear mixed-integer model, SCIP’s exact search is appropriately described as *branch-and-cut*. Presolve fixes linked skill variables, removes redundant rows, and tightens the remaining domains. SCIP then solves LP relaxations to obtain dual bounds, branches on fractional action decisions, separates valid cutting planes, propagates bound changes through the constraint system, and applies primal heuristics to obtain feasible action sets. Search continues until the primal–dual gap is closed and optimality is certified, or until SCIP returns another termination status. The implementation reports that status explicitly before any decoded action is used.

Recorded environment-synthesis solve. The recorded run contains 100 seeds and six projection–direction candidates per seed, giving 600 primary action variables. It requests 100 synthesized environments, uses active coverage slack cap $\bar{\xi} = 0.2$, and PySCIPOpt reports `status=optimal` with objective value 49.9104. Since the target count equals the seed count, the run still exports one selected action per seed. The coverage slack records bounded shortfall for active skill targets and conditions the decoded skill payload; it does not change the downstream export size of 100 verified environments. Alternative portfolio settings with smaller N or tighter coverage slack would create a stronger cross-seed allocation problem.

Solution decoding and audit trail. The implementation accepts the SCIP path only when PySCIPOpt reports `optimal`. Selected x and u variables are decoded with a 0.5 threshold, after which the action contract and induced skill subgraph are passed to the environment-synthesis prompt. The trace records the backend, formulation, status, objective value, candidate and target counts, requested coverage, realized slack, selected actions, and selected skill nodes. The objective value is a solver audit quantity for Eq. 3; downstream Pass@1 is measured separately after RL training.

Fallback and correctness boundary. For portability, an unavailable PySCIPOpt installation or a non-optimal SCIP status triggers a backend-explicit fallback: exact enumeration is attempted only when the candidate list contains at most 24 entries, and larger instances use a deterministic coverage-aware greedy policy. Every selected item retains the backend label, and the recorded planning trace uses

PySCIPOpt rather than either fallback. Solver optimality certifies the discrete selection problem defined by the input coefficients and constraints. Semantic consistency of the generated environment is certified separately by synchronized artifact materialization and the post-generation gold-solution verifier described in Section A.3.

C Prompt and Task-Instruction Excerpts

This section reproduces the prompt clauses that determine synthesis strategy and artifact consistency. For readability, we omit package-manager and mirror fallbacks, repeated JSON-schema boilerplate, task-specific file locations, and repeated output-format instructions. The excerpts retain the clauses that distinguish the compared strategies, define the synchronized-artifact contract, and specify task semantics.

C.1 Shared Baseline Synthesis Contract

The four prompting baselines call the same artifact-generation template. They differ only in the strategy and evolution-direction fields injected after the shared contract.

Shared baseline prompt (core excerpt)

Synthesize one Terminal-Bench-style task directory from the seed task. Return exactly one JSON object, with top-level keys `task_name`, `instruction_md`, `files`, and `metadata`.

- Create a small but real task variant rather than restating the seed.
- Rewrite the user-facing instruction and synchronize the oracle solution, test runner, verifier tests, execution environment, and any changed fixtures or helper files.
- Keep the container build context valid, pin the runtime, and ensure that every command used by the solution and tests exists in the generated environment.
- Make the tests verify the generated instruction and solution. Every tested threshold, tie-break, filename, output key, or fixture value must be stated in the instruction rather than hidden in the verifier.
- Make the test runner write reward 1 on success and reward 0 on failure, including when the underlying test command fails.
- Before returning, mentally execute the build, oracle solution, and verifier; the expected result is reward 1.

The runtime suffix supplies the selected strategy instruction, evolution direction, sample index, synthesis model, seed name, and shortened seed files for structural reference.

Baseline-specific strategy injection

few_shot; direction: none. “Create a close variant that preserves the seed task structure while changing concrete requirements and verifier expectations.”

self_instruct; direction: none. “Invent a related terminal task in the same domain with its own task description, solution, and verifier.”

evol_instruct_in_depth; direction: in_depth. “Deepen the seed by adding a harder constraint or edge case and update solution and tests accordingly.”

evol_instruct_in_breadth; direction: in_breadth. “Broaden the seed into a neighboring task type while keeping the same Terminal-Bench directory contract.”

The shared template controls artifact completeness, whereas the injected sentence fixes how the baseline moves from the seed. Few-shot stays structurally close, Self-Instruct permits a related same-domain task, and the two Evol-Instruct variants commit to depth or breadth before observing the policy-relative reward band.

Shared baseline repair prompt (core excerpt)

Repair a failed generated Terminal-Bench-style task directory. Return one complete JSON object rather than patch fragments.

- Make instruction_md identical to the generated instruction file after trimming whitespace.
- Make the oracle solve exactly that instruction and make both verifier layers test exactly that instruction and oracle behavior.
- Include every changed fixture or helper file, preserve a buildable environment context, and pin the runtime.
- State exact verifier expectations in the instruction and always emit reward 0 or 1 even when the test command fails.
- Re-evaluate the build-solve-test path and target oracle reward 1.

The repair call receives the strategy, evolution direction, repair index, failure summary, previous artifact JSON, and seed files. Repair preserves the baseline policy while correcting parsing, build, static-validation, or oracle-verification failures.

C.2 Solver-Conditioned Envs-FORGE Prompt

Envs-FORGE retains the same complete-environment output contract but conditions generation on an optimized action. The prompt receives a solver payload, a projection instruction, an evolution-direction instruction, an artifact edit mask, and a skill subgraph before it sees the seed files.

Solver payload and Envs-FORGE objective (core excerpt)

The payload records the seed pass-rate estimate and reward band, the selected projection, evolution_direction, and combined action_variant, together with the projected pass rate, frontier score, and target difficulty. The prompt objective is:

- project the seed toward the current policy’s learning frontier;
- preserve the target skill subgraph rather than jumping to an unrelated easy task;
- increase easy seeds, reduce hard seeds into bridge tasks, and diversify frontier seeds near their current difficulty;
- materialize the selected action as one synchronized executable environment.

The six possible action variants are the Cartesian product of increase_complexity, reduce_complexity, or diversify_frontier with in_depth or in_breadth.

Projection and evolution-direction instructions

increase_complexity. The seed appears easy. Add one or two concrete constraints, edge cases, stricter output requirements, or a larger fixture while preserving the seed skill subgraph.

reduce_complexity. The seed appears beyond the current agent. Split away secondary systems or create a smaller bridge task while preserving the target skill subgraph and its important artifacts.

diversify_frontier. The seed is near the frontier. Keep difficulty similar, diversify data and constraints, and avoid large simplification or unrelated breadth jumps.

in_depth. Strengthen constraints and edge cases without changing the core skill graph.

in_breadth. Vary fixtures, neighboring constraints, or surrounding context while keeping difficulty in the same neighborhood.

Artifact edit masks (core operators)

Let INST, DATA, SOL, TEST, ENV, and HELP denote the instruction, fixtures, oracle solution, tests, environment, and helper files.

increase_complexity. Required: INST, DATA, SOL, TEST; optional: ENV, HELP. Operators add explicit constraints and edge cases, change or enlarge fixtures, update the algorithm, and strengthen checks for boundary cases and deterministic outputs.

reduce_complexity. Required: INST, DATA, SOL, TEST, ENV; optional: HELP. Operators split secondary systems, replace live dependencies with deterministic fixtures, rewrite the solution for a bridge task, test the core skill only, and simplify the runtime.

diversify_frontier. Required: INST, DATA, SOL, TEST; optional: ENV, HELP. Operators reframe at similar difficulty, replace fixtures while preserving scale, update expected logic while retaining the solution shape, and preserve verifier granularity.

Skill-subgraph and generation contract

The skill-subgraph prompt exposes three sets. `preserve_nodes` must remain observable in the generated instruction and verifier; additions must come from `allowed_added_nodes`; and a reduction may remove or simulate `allowed_removed_nodes` only when the preserved core remains testable. The model must treat every artifact in `required_artifacts` as a mandatory synchronized edit and restrict each edit to the operator families in the selected mask.

The remaining clauses require a compact runnable artifact, favor reuse of the seed runtime, permit deterministic file-level simulations for reduced system tasks, prohibit hidden test constraints, and require the same build-oracle-verifier mental execution used by the baselines. The returned metadata stores the solver payload alongside the target difficulty and projection summary.

The difference from a fixed baseline appears upstream of generation: a baseline injects one predetermined strategy sentence, whereas Envs-FORGE injects a policy-relative solver decision plus explicit skill and artifact constraints. Both families are held to the same executable-environment and gold-verification standard.

C.3 Original-to-Synthesized Instruction Excerpts

The following five pairs retain the clauses that define the task, its edge cases, and its verifier-facing outputs. Repeated file locations and delivery boilerplate are omitted. The colors match Figure 5.

Case I: Bash log processor → JSON log processor

Original instruction, core clauses.

- Fix a Bash script that recursively processes every log file, records each filename and line count in a global output and per-directory summaries, and sorts by filename.
- Make all updates atomic, restartable, deterministic, and safe under repeated or concurrent execution; process each file exactly once and handle spaces or special characters in filenames.

Synthesized instruction, core clauses.

- Replace plain-text outputs with a processed JSON array, one JSON summary per directory, and a top-level directory-count summary.
- Recursively handle nested and unusual filenames, correctly count empty and non-newline-terminated files, sort under the C locale, write through temporary-file replacement, protect updates with a file lock, and remain idempotent.

Case II: multi-format merger → deterministic sorted merger

Original instruction, core clauses.

- Read one CSV, one JSONL file, and one JSON file with potentially non-standard schemas; output email, phone, and status for every unique email in the union.
- Do not print complete inputs, and emit a standard three-column CSV with a header.

Synthesized instruction, core clauses.

- Preserve the union semantics but require deterministic ascending email order and the exact header email,phone,status.
- Represent missing values as empty strings and use standards-compliant CSV quoting for commas, quotes, and newlines; the output must remain deterministic across runs.

Case III: systemd log monitoring → log-analyzer bridge

Original instruction, core clauses.

- Deploy a continuously running JSON-logging application as a systemd service, route its journal through rsyslog, rotate the resulting files, and provide a health-monitoring script.
- Detect restart loops and errors over a recent time window, emit a dynamic JSON report, and validate service management, routing, permissions, rotation, and end-to-end integration.

Synthesized instruction, core clauses.

- Analyze deterministic newline-delimited JSON fixtures instead of a live service stack. Detect more than three restart events in a five-minute window, count warnings and errors, and rank the five most recent errors.
- Record malformed JSON or missing fields, handle empty and unreadable fixtures without crashing, and emit one deterministic structured report using only the standard library.

Case IV: full-stack token service → file-level token validator

Original instruction, core clauses.

- Implement a Java/Spring service backed by H2 that issues access, refresh, and one-time WebSocket tokens with different lifetimes and storage semantics.
- Rotate refresh tokens atomically, ensure exactly one success under concurrent renewal or consumption, revoke all user tokens on logout, and periodically delete expired, revoked, or consumed state.

Synthesized instruction, core clauses.

- Read deterministic configuration, token-event records, and a fixed reference time; compute expiry, revocation, and one-time-token consumption status.
- Repair eligible recently used expired tokens, assign risk scores for expired, revoked, consumed, and high-risk-user conditions, then sort by descending risk and ascending token identifier in a structured JSON output.

Case V: PGN repair and puzzles → diversified PGN repair

Original instruction, core clauses.

- Repair multiple corrupted PGN games by detecting illegal, missing, and ambiguous moves; identify critical positions and tactical patterns.
- Solve FEN-based puzzles, including mate-in- N instances, and return repaired games, puzzle continuations, and aggregate analysis.

Synthesized instruction, core clauses.

- Repair one corrupted game containing exactly one illegal move, 15. Kf9, whose required correction is 15. Kf1.
- Validate every move with python-chess, preserve all PGN headers, and return the original PGN, repaired PGN, and a structured error record. No other error is present.

All five synthesized instructions were materialized with synchronized fixtures, oracle solutions, tests, and environments. Static validation passed and each oracle obtained reward 1. As in Figure 5, this establishes internal consistency of the generated task bundles; downstream policy accuracy is reported in the benchmark results.

D Compute Resources

For each prompt, we sample 8 rollouts with temperature 1.0 and top- $p = 0.9$. The training batch size, PPO mini-batch size, and per-GPU micro-batch size are all set to 1. Both the maximum prompt length and maximum response length are set to 8192 tokens. Each trajectory allows at most 50 agent-environment interaction steps, with a trajectory timeout of 900 seconds. The actor learning rate is set to 1×10^{-6} . We train for one epoch, enable automatic checkpoint resume, and save a checkpoint at every step; after training finishes, only the final actor model weights are retained. Rewards are computed solely from Terminal-Bench test outcomes, with test reward weight 1 and judge reward weight 0. We do not include a KL term in the reward. Training is conducted on 2 H800 80GB GPUs. The actor, reference policy, and rollout engine share the same visible GPUs. We use FSDP2 for distributed training, with gradient checkpointing, activation offloading, and FSDP offload policy enabled to reduce GPU memory usage. Rollouts are generated asynchronously by the vLLM hybrid engine with tensor parallel size 2. The model weights use bfloat16 precision, the KV cache uses FP8, and both the maximum model length and maximum number of batched tokens are set to $32k$.

The current runs require two H800 80 GB GPUs

with Fully Sharded Data Parallel (FSDP) and CPU offload for training, plus one H800 80 GB GPU for evaluation. Environment synthesis also requires candidate generation, MILP selection, container builds, and executable verification. Larger seed pools or larger backbones will increase synthesis, training, and evaluation cost.